Model prediction

Model prediction

1. Maximum performance mode
    1.1. Enable the MAX mode
    1.2. Enable the Jetson clocks
2. Model prediction
    2.1. The CLI use
    2.2. The Python use
        2.2.1. The USB camera
            Preview of the effect

1. Maximum performance mode

1.1. Enable the MAX mode

Enabling the MAX Power Mode (the maximum power mode) on the Jetson ensures that all the CPU and GPU cores are turned on:

```
sudo nvpmodel -m 0
```

1.2. Enable the Jetson clocks

Enabling the Jetson Clocks ensures that all the CPU and GPU cores run at the maximum frequency:

```
sudo jetson_clocks
```

2. Model prediction

Note: this example requires that the "Model training and conversion" course has been run first to generate the model file before it can be used.

2.1. The CLI use

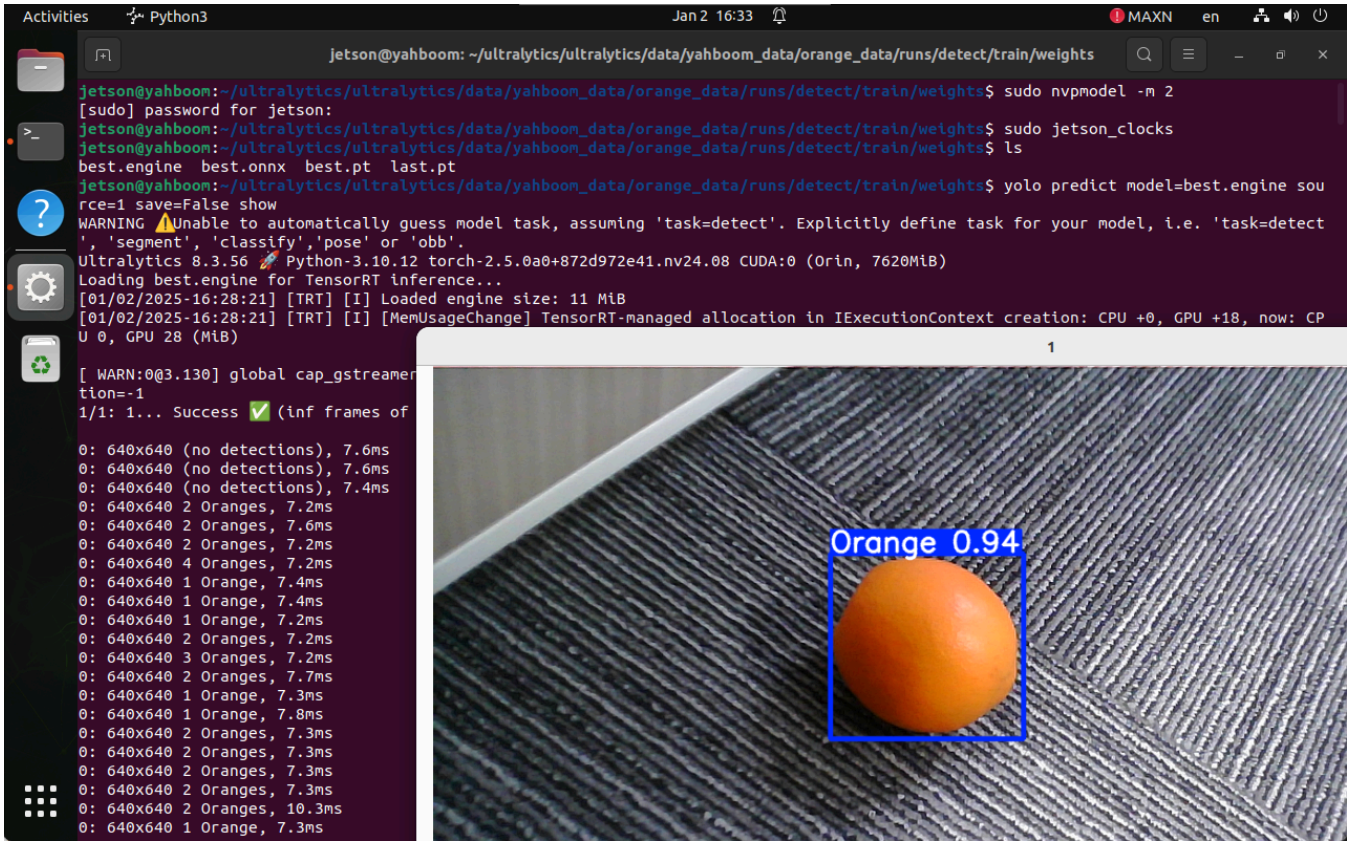
The CLI currently only supports calling the USB camera

Open a terminal and enter the path where the previous course generated the model

```
cd
/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights
```

Start the program

```
yolo predict model=best.engine source=0 save=False show
```



2.2. The Python use

Use Python to call the USB camera to recognize the oranges.

2.2.1. The USB camera

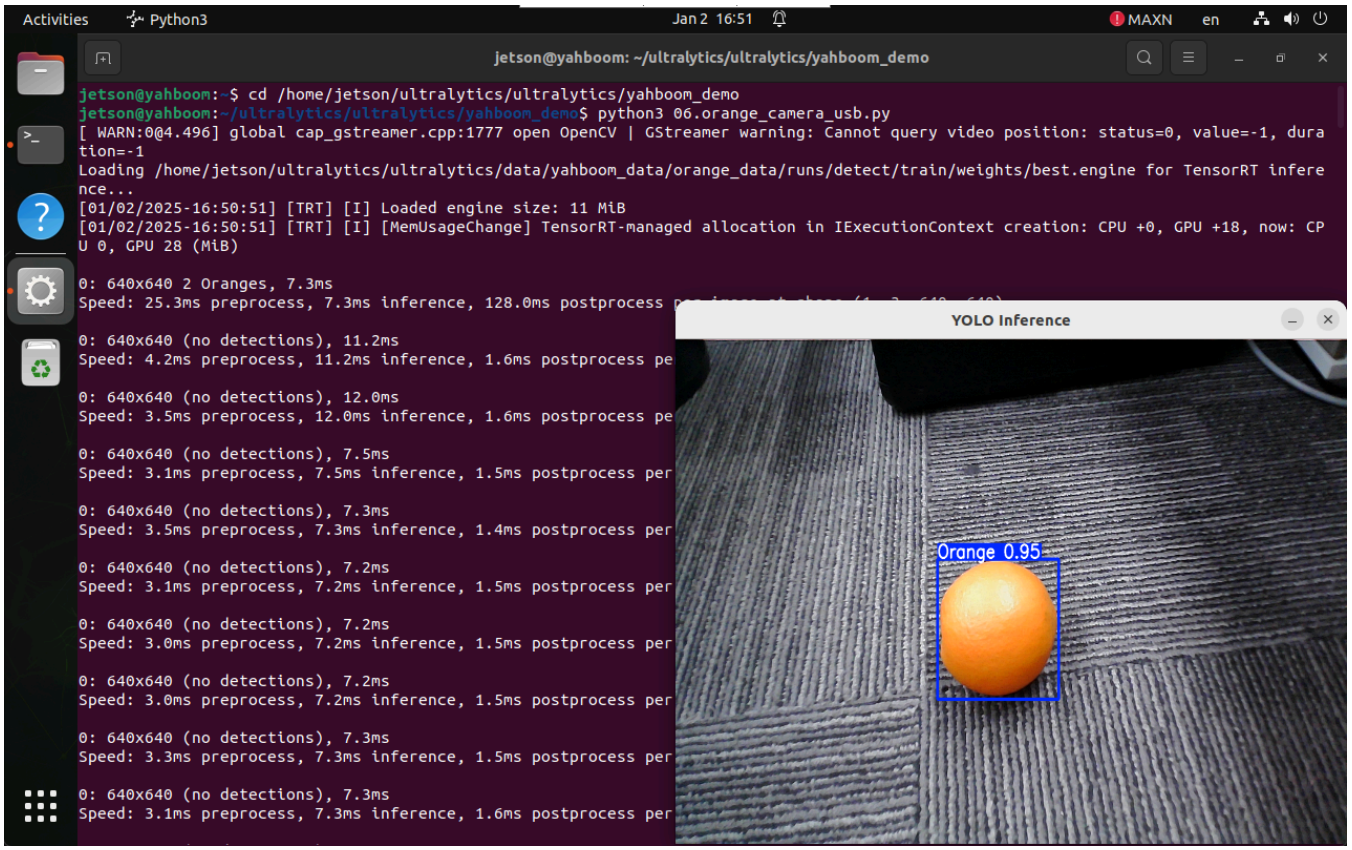
Use best.engine to predict the camera image:

```
cd /home/jetson/ultralytics/ultralytics/yahboom_demo
```

Run the code: click the preview window and press the q key to terminate the program!

```
python3 06.orange_camera_usb.py
```

Preview of the effect



The sample code:

```
import cv2
from ultralytics import YOLO

# Load the YOLO model
# model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.pt")
# model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.onnx")
model =
YOLO("/home/jetson/ultralytics/ultralytics/data/yahboom_data/orange_data/runs/detect/train/weights/best.engine")

# Open the cammera
cap = cv2.VideoCapture(0)

# Get the video frame size and frame rate
frame_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
frame_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

# Define the codec and create a VideoWriter object to output the processed video
output_path = "/home/jetson/ultralytics/ultralytics/output/06.orange_camera_usb.mp4"
fourcc = cv2.VideoWriter_fourcc(*'mp4v')  # You can use 'XVID' or 'mp4v' depending on your
platform
```

```
out = cv2.VideoWriter(output_path, fourcc, fps, (frame_width, frame_height))

# Loop through the video frames
while cap.isOpened():
    # Read a frame from the video
    success, frame = cap.read()

    if success:
        # Run YOLO inference on the frame
        results = model(frame)

        # Visualize the results on the frame
        annotated_frame = results[0].plot()

        # Write the annotated frame to the output video file
        out.write(annotated_frame)


        # Display the annotated frame
        cv2.imshow("YOLO Inference", cv2.resize(annotated_frame, (640, 480)))

        # Break the loop if 'q' is pressed
        if cv2.waitKey(1) & 0xFF == ord("q"):
            break
    else:
        # Break the loop if the end of the video is reached
        break
# Release the video capture and writer objects, and close the display window
cap.release()
out.release()
cv2.destroyAllWindows()
```

Note: the path that points to the model in the code, model = YOLO, must be replaced with your own latest path if the model data is updated by retraining.